RELATÓRIO TÉCNICO

Biblioteca modular em C para grafos

Disciplina:	MAC0216
Repositório:	https://github.com/ChuanChenUsp/biblioteca-grafo-c
Data:	19 de outubro de 2026

Sumário

1 Descrição do problema e do domínio escolhido **1**

2 Arquitetura do software **1**

3 Organização e responsabilidade dos módulos **2**
3.1 Core . . . 2
3.2 Edges . . . 2
3.3 Properties . . . 2
3.4 Traversal . . . 2

4 Descrição da API pública **2**

5 Descrição da aplicação cliente **3**
5.1 Problemas resolvidos . . . 4
5.2 Execução . . . 5
5.3 Funções da API usadas pelo cliente . . . 6

6 Processo de compilação utilizando Makefile **6**
6.1 Alvos do Makefile . . . 7

7 Estratégia de compilação incremental **7**

8 Geração e utilização da biblioteca estática **7**
8.1 Geração . . . 8
8.2 Utilização pela aplicação . . . 8
8.3 Verificação . . . 8

9 Geração e utilização da biblioteca dinâmica **9**
9.1 Geração . . . 9
9.2 Utilização pela aplicação . . . 9
9.3 Evidência de uso em tempo de execução . . . 10

10 Comparação entre ligação estática e dinâmica **10**
10.1 Resultados medidos no projeto . . . 11
10.2 Quando usar cada uma . . . 11

11 Estratégia e resultado dos testes **11**
11.1 Estratégia . . . 11
11.2 Casos cobertos . . . 12
11.3 Execução e resultados . . . 12

12 Teste que falhou e posterior correção **13**
12.1 Origem do defeito . . . 13

12.2 Teste que falha 13
12.3 Correção 14
12.4 Resultado 14

13 Estratégia de versionamento utilizando Git **15**

14 Descrição da branch, merge e tag `v1.0` **15**

15 Dificuldades encontradas e decisões de projeto **16**
15.1 Decisões de projeto 16
15.2 Dificuldades encontradas 17

16 Resultados obtidos e possíveis melhorias **18**
16.1 Resultados obtidos 18
16.2 Possíveis melhorias 18

1. Descrição do problema e do domínio escolhido

O domínio escolhido é **teoria dos grafos**. O projeto consiste no desenvolvimento de uma biblioteca modular em C para representar e manipular grafos. Grafos são estruturas matemáticas formadas por vértices e arestas, utilizadas para modelar relações entre objetos, como redes de computadores, mapas, relações sociais e dependências entre tarefas.

A biblioteca busca encapsular operações comuns sobre grafos em uma interface reutilizável. Dessa forma, programas clientes podem utilizar as funcionalidades oferecidas sem precisar conhecer todos os detalhes internos da representação dos dados.

A implementação utiliza uma estrutura de grafo que armazena o número de vértices, o número de arestas, uma matriz de adjacência. A matriz permite consultar a relação entre os pares de vértices.

2. Arquitetura do software

A arquitetura foi organizada em módulos com responsabilidades distintas: núcleo de representação do grafo, operações sobre arestas, propriedades do grafo e aplicação cliente. Os arquivos de cabeçalho expõem declarações e tipos necessários aos demais módulos; os arquivos `.c` contêm as implementações.

Arquivo	**Responsabilidade**
`core.h / core.c`	Definição das estruturas e criação/liberação do grafo.
`edges.h / edges.c`	Realiza perações relacionadas às arestas.
`properties.h / properties.c`	Consultas e propriedades estruturais do grafo.
`traversal.h / traversal.c`	Percorrer o grafo e verificar alcançabilidade
`main.c`	Programa cliente utilizado para demonstrar ou testar a biblioteca.
`Makefile`	Regras de compilação, ligação e limpeza dos artefatos.

A separação favorece a manutenção e permite que novas operações sejam adicionadas sem concentrar toda a implementação em um único arquivo. A interface pública deve expor apenas os tipos e as funções necessárias ao cliente.

3. Organização e responsabilidade dos módulos

3.1 Core

O módulo `core` declara o tipo `Graph` e concentra a criação e a gestão da memória associada à estrutura. A representação inclui os campos `n`, `m` e `adj`. A criação aloca a estrutura principal e inicializa a matriz de adjacência. Em caso de falha de alocação, a implementação deve liberar a memória já reservada antes de retornar erro.

3.2 Edges

O módulo `edges` concentra funções relacionadas à estrutura `Edge`, cujos campos `u` e `v` identificam os extremos da aresta e `w` pode representar seu peso.

3.3 Properties

O módulo `properties` reúne funções que consultam características do grafo. Por exemplo, a função `graph_is_connected` verifica se todos os vértices pertencem à mesma componente conexa. A definição final da função deve esclarecer se o comportamento é destinado a grafos não direcionados e como trata um grafo vazio.

3.4 Traversal

O módulo traversal é responsável pelo percurso do grafo a partir de um vértice de origem, com o objetivo de responder a questões sobre a relação entre vértices: quais podem ser alcançados, quantos pertencem à mesma componente conexa e qual a distância que os separa. Trata-se de um módulo exclusivamente de consulta, que não cria, modifica nem destrói o grafo recebido.

4. Descrição da API pública

O cliente inclui apenas `graph.h`, que reúne os quatro cabeçalhos dos módulos.

Função	**Descrição**	**Retorno em erro**
`Graph *graph_create(unsigned n)`	Cria grafo com n vértices e sem arestas	`NULL` (falta de memória)
`void graph_destroy(Graph *g)`	Libera o grafo; aceita `NULL`	–
`int graph_get_vertices(Graph *g)`	Retorna o número de vértices do grafo	`0`
`bool graph_add_edge(g, u, v, w)`	Insere/substitui a aresta u–v com peso w; atualiza o contador `m`	`false`
`bool graph_remove_edge(g, u, v)`	Remove a aresta (equivale a inserir peso 0)	`false`

Função	Descrição	Retorno em erro
`int graph_get_weight(g, u, v)`	Peso da aresta	`0` (também quando não há aresta)
`Edge *graph_get_edges(g, int *count)`	Vetor com as arestas ($u \leq v$); o chamador faz `free`	`NULL`
`int graph_degree(g, u)`	Grau do vértice	`-1`
`bool graph_is_reachable(g, u, v)`	v é alcançável a partir de u (DFS)	`false`
`int graph_component_size(g, u)`	Tamanho da componente conexa de u	`0`
`int graph_distance_length(g, u, v)`	Menor distância em número de arestas (BFS)	`-1`
`int graph_distance_weight(g, u, v)`	Menor distância ponderada (Dijkstra)	`-1`
`int *graph_distance_length_all(g, u)`	Distâncias em arestas de u a todos	`NULL`
`int *graph_distance_weight_all(g, u)`	Distâncias ponderadas de u a todos	`NULL`
`bool graph_is_connected(g)`	O grafo é conexo (grafo vazio é conexo)	`false`
`bool graph_has_cycle(g)`	Existe ciclo (laço conta como ciclo)	`false`
`bool graph_is_tree(g)`	Conexo e sem ciclos	`false`
`bool graph_is_forest(g)`	Sem ciclos	`false`
`bool graph_is_bipartite(g)`	Admite 2-coloração (DFS iterativa)	`false`

5. Descrição da aplicação cliente

A aplicação cliente está em `app/main.c` e demonstra o uso da biblioteca na resolução de três problemas clássicos de grafos, do tipo proposto em plataformas de programação competitiva. Cada problema é descrito no código por seu enunciado, pela *modelagem* (como a entrada vira um grafo) e pela solução, e exercita um módulo diferente da biblioteca.

O cliente respeita a separação entre interface e implementação exigida pelo enunciado:

- inclui apenas `graph.h`, o cabeçalho público que reúne os módulos;
- não inclui nenhum arquivo `.c` da biblioteca;

- não acessa a estrutura interna do grafo: o número de vértices é obtido por `graph_get_vertices`, e não por `g->n`;
- é compilado separadamente e ligado à biblioteca estática (`bin/graph_static`) e à biblioteca dinâmica (`bin/graph_shared`), a partir do mesmo `main.c`.

5.1 Problemas resolvidos

Problema	**Módulo principal**	**Funções usadas**
Is it a tree?	`properties`	`graph_create`, `graph_add_edge`, `graph_remove_edge`, `graph_is_tree`, `graph_destroy`
Tree Diameter	`traversal`	`graph_distance_length_all`, `graph_get_vertices`
Graph and String	`core` e `edges`	`graph_degree`, `graph_get_weight`, `graph_get_vertices`

Problema 1: *Is it a tree?*

Enunciado resumido. Dado um grafo não direcionado, informar se ele é uma árvore, isto é, se é conexo e não tem ciclos.

Modelagem. Cada vértice do problema é um vértice do grafo, e cada aresta `u v` vira `graph_add_edge(g, u, v, 1)`.

Solução. A função `graph_is_tree` do módulo `properties` realiza exatamente essa verificação. O programa a aplica a um caminho $0-1-2-3$ (é árvore), depois à mesma estrutura com a aresta $3-0$, que fecha um ciclo (não é árvore), e novamente após remover essa aresta com `graph_remove_edge` (volta a ser árvore). O exemplo exercita, assim, a inserção e a remoção de arestas e a mudança do resultado da propriedade.

Problema 2: *Tree Diameter*

Enunciado resumido. Dada uma árvore com N nós, determinar o diâmetro, a maior distância, em número de arestas, entre dois nós.

Modelagem. Os nós $1..N$ viram os vértices $0..N-1$, e cada aresta `a b` vira `graph_add_edge(g, a-1, b-1, 1)`.

Solução. São feitas duas buscas em largura, ambas com `graph_distance_length_all`, função do módulo `traversal` que devolve as distâncias de uma origem a todos os vértices:

1. a partir do vértice 0, encontra-se o vértice a mais distante;
2. a partir de a, a maior distância encontrada é o diâmetro.

O método funciona porque, em uma árvore, o vértice mais distante de qualquer vértice

é uma das extremidades de algum caminho de comprimento máximo. O exemplo usa a árvore com arestas $1-2$, $1-3$, $3-4$ e $3-5$, cujo diâmetro é 3.

Problema 3: *Graph and String*

Enunciado resumido. Dado um grafo simples com N vértices, decidir se existe uma string de N letras em `a`, `b` e `c` tal que dois vértices distintos i e j estão ligados por aresta se, e somente se, `s[i]` e `s[j]` são iguais ou vizinhas no alfabeto (`a`–`b` e `b`–`c` são vizinhas; `a` e `c` não). A resposta é `Yes` e uma string válida, ou `No`.

Modelagem. O vértice i representa a posição i da string, e a aresta representa um par de posições compatíveis. A consulta de adjacência é feita com `graph_get_weight(g, i, j) != 0`.

Solução.

1. todo vértice de grau $N-1$ (consultado com `graph_degree`) pode receber `b`, pois `b` é compatível com qualquer letra;
2. o primeiro vértice restante recebe `a`; os demais recebem `a` se forem vizinhos dele e `c` caso contrário;
3. todos os pares são conferidos: deve haver aresta se, e somente se, as letras são iguais ou uma delas é `b`.

O exemplo tem o vértice 0 ligado a todos e os pares $1-2$ e $3-4$ ligados entre si, resultando na string `baacc`.

5.2 Execução

A aplicação pode ser executada com a biblioteca dinâmica (`make run` ou `make run-shared`, que define `LD_LIBRARY_PATH=lib`) ou com a biblioteca estática (`make run-static`). A saída é a mesma nos dois casos:

```
=== Problema 1: Is it a tree? ===
caminho 0-1-2-3:      YES
com a aresta 3-0:     NO
sem a aresta 3-0:     YES

=== Problema 2: Tree Diameter ===
diametro: 3

=== Problema 3: Graph and String ===
Yes baacc
```

Os resultados coincidem com os valores esperados indicados nos comentários do código. As evidências de que cada executável usa a biblioteca correspondente estão nas Seções 8 e 9.

5.3 Funções da API usadas pelo cliente

O cliente usa nove funções da biblioteca: `graph_create`, `graph_destroy`, `graph_add_edge`, `graph_remove_edge`, `graph_get_weight`, `graph_degree`, `graph_is_tree`, `graph_distance_length_all` e `graph_get_vertices`. As outras dez funções públicas são exercitadas pelo programa de testes (Seção 11); somando cliente e testes, todas as 19 funções públicas da biblioteca são utilizadas.

6. Processo de compilação utilizando Makefile

A compilação do projeto é automatizada por um único Makefile, localizadona raiz do repositório. O make refaz um alvo somente quando ele n~ao existe ou quando alguma de suas dependências é mais recente que ele, o que sustenta a compilação incremental.

O processo de compilação é dividido em cinco etapas, cada uma representada por uma regra:

1. Compilação dos módulos. Cada arquivo src/modulo.c é compilado separadamente, sem ligação (-c), e gera build/modulo.o. Os objetos usam -fPIC para poderem compor a biblioteca dinâmica.

2. Biblioteca estática. Os quatro objetos são empacotados em lib/libgraph.a pelo utilitário ar.

3. Biblioteca dinâmica. Os mesmos objetos são reunidos em lib/libgraph.so com gcc -shared.

4. Aplicação cliente. app/main.c é compilado e ligado à biblioteca com -Llib -lgraph, gerando bin/graph app.

5. Testes. tests/graph test.c é compilado e ligado da mesma forma, gerando bin/graph test.

A relação de dependência entre os arquivos á a seguinte:

```
bin/graph_app  <--  app/main.c  +  lib/libgraph.so
bin/graph_test <--  tests/graph_test.c  +  lib/libgraph.so

lib/libgraph.a   <--  build/*.o   (ar rcs)
lib/libgraph.so  <--  build/*.o   (gcc -shared)

build/core.o        <--  src/core.c        include/core.h
build/edges.o       <--  src/edges.c       include/edges.h  include/core.h
build/traversal.o   <--  src/traversal.c   include/traversal.h  include/core.h
build/properties.o  <--  src/properties.c  include/properties.h  include/core.h
```

6.1 Alvos do Makefile

Os alvos do makefile seguem a seguinte organização:

Alvo	Ação
`all`	Alvo padrão. Gera `lib/libgraph.a`, `lib/libgraph.so` e `bin/graph_app`
`run`	Executa a aplicação com `LD_LIBRARY_PATH=lib ./bin/graph_app`
`test`	Compila `tests/graph_test.c` e executa `bin/graph_test`
`clean`	Remove o conteúdo de `build/`, `bin/` e `lib/`
`install`	Instala as bibliotecas e os cabeçalhos em um prefixo configurável (ver abaixo)

Os alvos `all`, `clean`, `run`, `test` e `install` estão declarados em `.PHONY`, pois não correspondem a arquivos: sem essa declaração, a existência de um arquivo com o mesmo nome impediria a execução do alvo.

7. Estratégia de compilação incremental

A compilação incremental recompila somente os arquivos que precisam ser atualizados. Para isso, o `Makefile` deve produzir arquivos-objeto (`.o`) e declarar suas dependências em relação aos arquivos-fonte e cabeçalhos.

Cada módulo tem uma regra própria que lista suas dependências: o `.c` e os cabeçalhos que ele inclui. Por exemplo, `build/edges.o` depende de `src/edges.c`, `include/edges.h` e `include/core.h`. O `make` compara a data de modificação dos arquivos e recompila só o que ficou desatualizado.

Experimento realizado. Depois de uma compilação completa, o comando `touch src/edges.c` seguido de `make` executou apenas:

```
gcc -Wall -Iinclude -fPIC -c src/edges.c -o build/edges.o
ar rcs lib/libgraph.a build/core.o build/edges.o build/traversal.o build/
    properties.o
gcc -shared -o lib/libgraph.so build/core.o build/edges.o build/traversal.o
    build/properties.o
```

Os objetos `core.o`, `traversal.o` e `properties.o` não foram recompilados. As duas bibliotecas foram regeneradas porque dependem de `edges.o`.

8. Geração e utilização da biblioteca estática

Uma biblioteca estática é um arquivo que reúne objetos já compilados. Na ligação, o `gcc` copia para o executável os objetos de que o programa precisa, de modo que o resultado não depende mais da biblioteca em tempo de execução.

8.1 Geração

A biblioteca `lib/libgraph.a` é produzida pelo utilitário `ar` a partir dos quatro objetos dos módulos, pela regra `$(LIB_STATIC): $(OBJ)` do `Makefile`:

```
ar rcs lib/libgraph.a build/core.o build/edges.o build/traversal.
   o build/properties.o
```

As opções `rcs` criam o arquivo, inserem os objetos e gravam o índice de símbolos usado pelo ligador. O conteúdo pode ser conferido com `ar -t`:

```
$ ar -t lib/libgraph.a
core.o
edges.o
traversal.o
properties.o
```

Como a regra depende de todos os objetos, a biblioteca é refeita sempre que um módulo muda.

8.2 Utilização pela aplicação

O `Makefile` liga o cliente à biblioteca estática informando o caminho do arquivo `.a` diretamente, o que garante que a versão dinâmica não seja escolhida:

```
gcc -Wall -Iinclude app/main.c lib/libgraph.a -o bin/graph_static
```

Como o cliente inclui apenas `graph.h` (a interface pública), nenhum arquivo `.c` da biblioteca participa dessa compilação; o código dos módulos entra no executável apenas na etapa de ligação, vindo do `.a`. O alvo `make run-static` compila (se necessário) e executa `bin/graph_static`; o alvo `make test` também gera e executa `bin/graph_test_static`, ligado da mesma forma.

8.3 Verificação

As verificações abaixo foram feitas com o cliente real, `bin/graph_static`:

Verificação	**Resultado**
`ldd bin/graph_static` lista `libgraph`?	Não
Execução sem `LD_LIBRARY_PATH`	Funciona
Execução após mover o diretório `lib/`	Funciona (saída `Yes baacc`)
Funções `graph_*` presentes no executável (`nm`)	19, todas as públicas
Tamanho do executável	25 592 bytes

Esses resultados mostram que o código da biblioteca foi incorporado ao executável, que pode ser distribuído e executado sem os arquivos de `lib/`.

Há um detalhe a notar: o cliente chama apenas nove funções, mas as dezenove aparecem no executável. Isso ocorre porque o ligador copia do arquivo `.a` *objetos inteiros*: como o

cliente usa funções dos quatro módulos, os quatro objetos são incorporados, com todas as funções que contêm.

9. Geração e utilização da biblioteca dinâmica

Uma biblioteca dinâmica (compartilhada) permanece em um arquivo separado do executável. Na ligação, o executável apenas registra que precisa dela; o código é carregado pelo sistema operacional quando o programa é executado.

9.1 Geração

Os objetos são compilados com `-fPIC`, que gera código independente de posição, pois a biblioteca pode ser carregada em endereços diferentes a cada execução. Em seguida, são reunidos com `-shared`:

```
gcc -Wall -Iinclude -fPIC -c src/edges.c -o build/edges.o     # (
   e os demais modulos)
gcc -shared -o lib/libgraph.so build/core.o build/edges.o build/
   traversal.o build/properties.o
```

O arquivo `lib/libgraph.so` tem 21 016 bytes. Os símbolos exportados podem ser listados com `nm`; o resultado são as 19 funções públicas:

```
$ nm -D --defined-only lib/libgraph.so | grep -c " T "
19
```

[**A COMPLETAR**] A regra do `Makefile` que gera a `.so` ainda inclui, no comando acima, a opção `$(SHAREFLAG)` (`-Llib -lgraph`). Ela é desnecessária ao criar a própria biblioteca e, com algumas configurações do ligador, faz o `libgraph.so` listar a si mesmo como dependência. Remova-a da regra para que o comando fique exatamente como mostrado.

9.2 Utilização pela aplicação

O cliente é ligado à biblioteca informando o diretório e o nome:

```
gcc -Wall -Iinclude app/main.c -Llib -lgraph -o bin/graph_shared
```

Quando existem `libgraph.a` e `libgraph.so` em `lib/`, a opção `-lgraph` faz o `gcc` escolher a versão dinâmica; por isso a ligação estática, na seção anterior, usa o caminho do arquivo `.a`.

Na execução, o carregador dinâmico precisa localizar o `.so`. Como ele fica em `lib/`, e não em um diretório do sistema, os alvos `make run` e `make run-shared` definem a variável `LD_LIBRARY_PATH`:

```
LD_LIBRARY_PATH=lib ./bin/graph_shared
```

9.3 Evidência de uso em tempo de execução

Como `main.c` chama funções da API, o ligador registra a dependência da biblioteca no executável. As verificações abaixo foram feitas com o cliente real, `bin/graph_shared`:

Verificação	**Resultado**
`readelf -d` (entradas `NEEDED`)	`libgraph.so` e `libc.so.6`
`ldd` sem `LD_LIBRARY_PATH`	`libgraph.so => not found`
Execução sem `LD_LIBRARY_PATH`	Falha: `error while loading shared libraries: libgraph.so: cannot open shared object file`
Execução com `LD_LIBRARY_PATH=lib`	Funciona (`make run`)
Tamanho do executável	16 456 bytes

A falha na execução sem a variável é a prova mais direta de que a biblioteca é carregada em tempo de execução: o programa não roda sem o arquivo `.so`. O mesmo vale para `bin/graph_test_shared`, gerado e executado por `make test`.

10. Comparação entre ligação estática e dinâmica

A diferença central entre as duas formas está no momento em que o código da biblioteca se junta ao programa. Na ligação estática, isso acontece na compilação e o código é copiado para o executável. Na ligação dinâmica, o executável guarda apenas uma referência à biblioteca, e o carregamento ocorre quando o programa é executado.

Aspecto	**Estática (`.a`)**	**Dinâmica (`.so`)**
Momento da ligação	Em tempo de compilação	Na carga do programa
Código da biblioteca	Copiado para o executável	Permanece no `.so`
Dependência na execução	Nenhuma	O `.so` precisa ser encontrado
Distribuição	Um único arquivo executável	Executável e `.so`
Atualização da biblioteca	Exige religar o cliente	Basta substituir o `.so`, se a interface for mantida
Opção de compilação	Nenhuma especial	`-fPIC` e `-shared`
Compartilhamento entre programas	Cada programa tem sua cópia	Uma cópia carregada serve a vários programas
Falha típica	Binário maior que o necessário	`error while loading shared libraries`

10.1 Resultados medidos no projeto

Os valores abaixo vêm do mesmo programa cliente (`tests/graph_test.c`) ligado das duas maneiras:

Medida	Estática	Dinâmica
Tamanho do executável	29 704 bytes	20 904 bytes
Tamanho da biblioteca	20 030 bytes (`.a`)	20 976 bytes (`.so`)
`libgraph` listada no `ldd`	Não	Sim (`not found` sem `LD_LIBRARY_PATH`)
Executa sem `LD_LIBRARY_PATH`	Sim	Não
Executa sem o diretório `lib/`	Sim	Não

O executável dinâmico é cerca de 8,8 KB menor. Essa economia, porém, é pequena neste projeto, e o arquivo `.so` de 20,9 KB continua necessário para executá-lo. Com um único cliente, o total em disco da versão dinâmica é maior. O ganho de espaço e de atualização só aparece quando vários programas usam a mesma biblioteca.

10.2 Quando usar cada uma

- **Estática:** indicada quando se deseja um executável autônomo, fácil de distribuir e sem risco de a biblioteca não ser encontrada, como em uma ferramenta de linha de comando entregue isoladamente.
- **Dinâmica:** indicada quando a biblioteca é usada por vários programas ou precisa ser atualizada sem recompilar os clientes, como ocorre com bibliotecas do sistema.

Para a `libgraph`, as duas formas são geradas pelo mesmo `make`, a partir dos mesmos objetos, e a escolha entre elas é feita apenas na ligação do cliente.

11. Estratégia e resultado dos testes

11.1 Estratégia

Os testes automatizados estão no arquivo `tests/graph_test.c`. Trata-se de um programa em C que usa apenas a interface pública (`graph.h`) e a macro `assert` da biblioteca padrão: cada verificação compara o resultado devolvido pela biblioteca com o valor esperado e interrompe o programa, com código de saída diferente de zero, se houver divergência.

O programa segue um **roteiro com estado**: constrói um grafo, altera-o em etapas e confere as propriedades esperadas após cada alteração. O grafo inicial tem 6 vértices e 5 arestas, formando uma árvore (caminho $3 - 1 - 0 - 2 - 4 - 5$):

```
graph_add_edge(g, 0, 1, 4);   graph_add_edge(g, 0, 2, 2);
graph_add_edge(g, 1, 3, 5);   graph_add_edge(g, 2, 4, 1);
graph_add_edge(g, 4, 5, 3);
```

O roteiro tem sete blocos, que também organizam o código do teste:

1. criação do grafo e verificação de que a alocação teve sucesso;
2. módulo `edges`: pesos, graus e listagem das arestas;
3. módulo `traversal`: alcançabilidade, tamanho de componente e distância em arestas;
4. módulo `properties` no grafo inicial (árvore): conexo, sem ciclo, árvore, floresta e bipartido;
5. criação de um ciclo com a aresta $(1, 2)$ e nova verificação das propriedades e dos graus;
6. remoção de arestas, que desconecta o grafo (a aresta $(2, 4)$) e depois desfaz o ciclo (a aresta $(1, 2)$);
7. entradas inválidas (vértice 6 em um grafo de vértices 0 a 5).

O teste contém **59 asserções**. Cada módulo da biblioteca tem pelo menos um caso específico: `core` (criação e liberação do grafo), `edges` (blocos 2, 5 e 6), `traversal` (blocos 3, 6 e 7) e `properties` (blocos 4, 5 e 6).

11.2 Casos cobertos

Tipo de caso	Exemplos
Uso normal	Peso e grau de vértices existentes; listagem das 5 arestas; alcançabilidade e tamanho de componente em grafo conexo; classificação do grafo como árvore, floresta ou bipartido
Mudança de estado	Após inserir a aresta $(1, 2)$: surgimento de ciclo, perda da propriedade de árvore, perda da bipartição, graus 3 e 3 e 6 arestas. Após remover $(2, 4)$ e $(1, 2)$: componentes de tamanhos 4 e 2 e retorno a uma floresta bipartida
Valor limite	Aresta inexistente (`graph_get_weight` devolve 0); vértices em componentes diferentes (distância -1)
Entrada inválida	Vértice 6 em `graph_degree`, `graph_distance_length`, `graph_distance_weight`, `graph_is_reachable` e `graph_component_size`

11.3 Execução e resultados

O alvo `make test` compila o teste, liga-o à biblioteca e o executa:

```
$(TEST): tests/graph_test.c
        $(CC) $(CFLAGS) tests/graph_test.c -Llib -lgraph -o $(
           TEST)

test: $(TEST)
        LD_LIBRARY_PATH=lib ./$(TEST)
```

Na versão atual do projeto, todas as asserções passam e o programa termina com código de saída 0. Durante a execução, o teste de entradas inválidas faz a biblioteca escrever em `stderr` mensagens como `6 não é vértice deste grafo!`. Se alguma asserção falhar, o programa é abortado e o `make` termina com erro.

12. Teste que falhou e posterior correção

Esta seção documenta um defeito real da biblioteca, descoberto ao testar valores no limite: o cálculo da distância ponderada estoura o tipo `int` quando a soma dos pesos é grande. O caso segue o ciclo pedido no enunciado: um teste escrito primeiro, que falha; a correção do código; e a nova execução, que passa.

12.1 Origem do defeito

Os pesos das arestas são do tipo `unsigned int`, mas as distâncias são devolvidas como `int`, com -1 indicando "sem caminho" ou erro. Nas funções `graph_distance_weight` e `graph_distance_weight_all` (`src/traversal.c`), o algoritmo de Dijkstra acumulava a soma em um `int`:

```
int nova = dist[x] + (int)g->adj[x][y];
if (nova < dist[y]) dist[y] = nova;
```

Cada peso isolado cabe em `int`, mas a soma pode não caber. Com duas arestas de pesos 1 500 000 000 e 1 000 000 000, a soma real é 2 500 000 000, maior que `INT_MAX` (2 147 483 647). Em C, esse estouro é comportamento indefinido para inteiros com sinal; na prática o valor "dá a volta": $2\,500\,000\,000 - 2^{32} = -1\,794\,967\,296$. A função devolvia esse número negativo como se fosse uma distância válida, e o chamador não tinha como distingui-lo de um resultado correto.

12.2 Teste que falha

Em uma branch própria, o primeiro commit acrescentou apenas o teste, sem alterar a biblioteca. O teste é um novo bloco (número 8) do programa `tests/graph_test.c`, posicionado depois de o grafo anterior ter sido liberado, o que permite reutilizar a variável `g`:

```
printf("8. Testar overflow em graph_distance_weight\n");

g = graph_create(3);

assert(graph_add_edge(g, 0, 1, 1500000000));
assert(graph_add_edge(g, 1, 2, 1000000000));

int distancia = graph_distance_weight(g, 0, 2);

printf("Distância com possível overflow: %d\n", distancia);

graph_destroy(g);
```

O grafo tem três vértices ligados em sequência, e o único caminho de 0 a 2 custa $2{,}5 \times 10^9$. Como esse valor não é representável em `int`, a decisão de projeto foi que a função devolva -1, o valor de erro que ela já usa, em vez de um número negativo arbitrário.

Com o código ainda sem correção, `make test` imprime o valor errado:

```
8. Testar overflow em graph_distance_weight
Distância com possível overflow: -1794967296
```

12.3 Correção

O segundo commit altera as duas funções de Dijkstra. Antes de somar, verifica-se se o novo custo caberia em `int`; se não couber, a aresta é ignorada. A comparação é feita sem estourar: em vez de calcular `dist[x] + peso`, compara-se `peso` com o espaço que resta, `INT_MAX - dist[x]`.

```
- int nova = dist[x] + (int)g->adj[x][y];
- if (nova < dist[y]) dist[y] = nova;
+ unsigned int peso = g->adj[x][y];
+ if (peso > (unsigned int)(INT_MAX - dist[x])) continue;   /* custo nao
    cabe em int */
+ int nova = dist[x] + g->adj[x][y];
+ if (nova < dist[y]) dist[y] = nova;
```

A subtração `INT_MAX - dist[x]` é segura porque `dist[x]` está sempre entre 0 e `INT_MAX`. Quando a aresta é ignorada, o vértice de destino permanece com distância infinita e é reportado como -1.

Por que ignorar a aresta em vez de encerrar a função. Uma primeira ideia foi devolver -1 imediatamente ao detectar o estouro. Isso erra quando há mais de um caminho. Por exemplo, com os caminhos $0 \to 1 \to 3$ (pesos $1{,}5 \times 10^9$ e $1{,}5 \times 10^9$, soma $3{,}0 \times 10^9$, que estoura) e $0 \to 2 \to 3$ (pesos $1{,}0 \times 10^9$ e $1{,}0 \times 10^9$, soma $2{,}0 \times 10^9$, que cabe), a distância correta de 0 a 3 é 2 000 000 000, pelo segundo caminho. O Dijkstra processa o vértice 1 antes de chegar ao 3 e, se abortasse ali, devolveria -1 sem ter descoberto o caminho válido. Ignorando apenas a aresta que estoura, a função encontra o caminho que cabe. Isso é seguro porque um caminho que não cabe em `int` custa mais que qualquer caminho que cabe, e, se o caminho mais barato cabe, todos os seus trechos também cabem e nenhuma de suas arestas é ignorada. Se todos os caminhos estouram, o resultado é -1.

12.4 Resultado

Após a correção, `make test` passa com as duas bibliotecas (dinâmica e estática), e o bloco novo imprime o valor esperado:

```
8. Testar overflow em graph_distance_weight
Distância com possível overflow: -1
```

A compilação segue sem avisos. O mesmo cenário, que antes devolvia $-1\,794\,967\,296$, passa a devolver -1.

13. Estratégia de versionamento utilizando Git

O código-fonte foi versionado com Git em um repositório hospedado no GitHub. A branch principal é a `main`, que deve conter sempre uma versão integrada do projeto.

O fluxo adotado pelo grupo foi o seguinte:

1. Cada módulo da biblioteca, ou cada frente de trabalho, foi desenvolvido em uma branch própria criada a partir da `main`.
2. Os commits foram feitos na branch, sem alterar diretamente a `main`.
3. Ao concluir uma etapa, a branch foi integrada à `main` por meio de um *Pull Request* no GitHub, que gera um commit de merge identificado por mensagens do tipo `Merge pull request #N from ...`.

Essa organização permitiu que integrantes diferentes trabalhassem em módulos distintos ao mesmo tempo (por exemplo, `traversal` e `properties`) sem sobrescrever o trabalho uns dos outros.

14. Descrição da branch, merge e tag `v1.0`

Ao longo do projeto, foram implementadas oito branches que, após revisão, foram incorporadas à branch principal por meio de merge. As branches utilizadas, suas respectivas finalidades e os pull requests (PRs) por meio dos quais foram integradas à são apresentados a seguir:

Branch	**Finalidade**	**Integração à `main`**
`core`	Esqueleto do projeto, tipos `Graph` e `Edge`, criação e destruição do grafo	Sem PR próprio; histórico incorporado pelo PR #2, pois a `edges` parte dela
`edges`	Módulo de arestas (`edges.c`, `edges.h`); `graph_get_edges` movida de `core.h`	PR #2 (`514409b`)
`solving_bugs`	Correções de compilação, tipos, memória e contagem de arestas; ajuste do *include guard*	PR #3 (`840b187`), #4 (`551510e`) e #12 (`1f73621`)
`properties`	Módulo de propriedades (ciclo, bipartição etc.) e sua documentação	PR #5 (`9cfc0b3`), #7 (`336c8b6`) e #11 (`ecf782b`)

Branch	Finalidade	Integração à `main`
`transversal`	Módulo `traversal`: alcançabilidade, distâncias e tratamento de overflow no Dijkstra	PR #6 (`4895d63`, módulo), #14 (`1b7316a`, nome de função), #15 (`8b6641b`, Doxygen em `traversal.h`), #20 (`83b148e`, primeira correção de overflow) e #21 (`b833088`, correção final)
`documentation`	Comentários Doxygen em `core.h` e `edges.h`	PR #9 (`19fc374`)
`main_file`	Cliente `app/main.c` (três problemas de programação competitiva) e cabeçalho unificado `graph.h`	PR #13 (`9e4985d`) e #16 (`9d86cba`)
`make-test`	`Makefile`, bibliotecas estática e dinâmica, `graph_get_vertices` e testes automatizados; teste do overflow	PR #10 (`b811ab4`) e #19 (`0c40520`)
`make`	Testes mais abrangentes (distâncias; `.so` e `.a`) e alvo `make install`	PR #17 (`4f542ef`)
`doxygen`	Configuração do Doxygen e documentação gerada	PR #18 (`42b66e7`)

Uma tag como `v1.0` identifica um ponto específico do histórico considerado uma versão estável. A tag v1.0 foi criada sobre o commit 83b148e, resultante da integração do PR # 20 (branch transversal). Ela marca uma versão estável intermediária.

15. Dificuldades encontradas e decisões de projeto

15.1 Decisões de projeto

Representação por matriz de adjacência. O grafo é guardado em uma matriz $n \times n$ de pesos (`unsigned int`), na qual o peso 0 indica a ausência de aresta. A escolha torna simples e rápida ($O(1)$) a consulta e a alteração de uma aresta e deixa os algoritmos fáceis de explicar. Em contrapartida, a memória cresce com n^2, e não é possível representar uma aresta de peso zero. Para o escopo do trabalho (grafos pequenos, usados em testes e demonstração), o grupo considerou a troca adequada.

Grafo não direcionado. Cada inserção ou remoção atualiza as duas posições simétricas da matriz. Laços são aceitos e contam 2 no grau do vértice, e o contador de arestas conta cada laço uma vez.

Divisão em quatro módulos por tipo de responsabilidade. `core` cuida do ciclo

de vida do grafo, `edges` altera e consulta arestas, `traversal` responde a perguntas sobre relações entre vértices a partir de uma origem e `properties` verifica propriedades do grafo inteiro. A função `graph_get_edges`, que estava em `core.h`, foi movida para `edges.h` (commit `0fd0f0f`), por pertencer à responsabilidade de arestas.

Escolha dos algoritmos.

- A distância em número de arestas usa busca em largura (BFS) com uma fila em vetor, que encontra o menor número de arestas sem considerar pesos.
- A distância ponderada usa o algoritmo de Dijkstra, adequado porque os pesos são não negativos. A seleção do próximo vértice é feita por busca linear ("simulação de fila de prioridade"), em vez de uma estrutura de heap: o custo é $O(n^2)$, compatível com a matriz de adjacência, e o código fica mais simples.
- `graph_is_reachable` e `graph_component_size` usam DFS recursiva; as propriedades em `properties.c` usam DFS iterativa com pilha explícita, o que evita estouro da pilha de chamadas em grafos grandes.

Interface pública enxuta. As funções auxiliares (`dfs_reach`, `dfs_component`, `dfs`, `dfs_has_cycle` e `new_edge`) são `static` e ficam restritas ao arquivo `.c` onde são usadas. O cliente inclui um único cabeçalho, `graph.h`, que reúne os quatro cabeçalhos dos módulos.

Tratamento de erros por valores sentinela. A biblioteca não encerra o programa em caso de erro. Para ponteiro nulo, vértice inválido ou falta de memória, cada função devolve um valor de erro conforme o seu tipo de retorno (`false`, -1, 0 ou `NULL`) e, em vários casos, escreve uma mensagem em `stderr`. A convenção está documentada nos comentários no estilo Doxygen de cada função.

15.2 Dificuldades encontradas

As principais dificuldades aparecem no histórico do Git. Elas se concentraram na integração dos módulos: em 11/10/2026 foram feitos 17 commits em um único dia, boa parte deles para corrigir problemas surgidos ao compilar e ligar o código escrito em partes separadas:

- **Erros de compilação e de tipos.** Ausência de `stdbool.h` (`f70bd14`), conflitos de tipos entre declarações em `.h` e definições em `.c` (`ade98df`) e avisos de comparação entre `int` e `unsigned int` (`1aa9e3b`).
- **Erros de memória.** Comportamentos indefinidos com ponteiros (`b35182b`) e vazamentos de memória (`6889b99`), como a liberação das linhas já alocadas quando uma alocação falha no meio da criação da matriz.
- **Contagem de arestas.** O contador de arestas ficava incorreto ao inserir e remover arestas, e os valores de retorno das funções também precisaram de correção (`5a4067f` e `2f5ee6e`). A solução final incrementa o contador somente quando uma aresta nova é criada e o decrementa somente quando uma aresta existente é removida; substituir o peso de uma aresta existente não o altera.

- **Coordenação entre integrantes.** Como cada módulo foi desenvolvido em uma branch, surgiram inconsistências de nomes e de inclusões (`cb89671`). Os nomes das funções de distância foram trocados mais de uma vez (`ebcc62c` e `46e6d5c`), e a ordem do `#define` do *include guard* em `edges.h` precisou de ajuste (`3e50bdf`).

16. Resultados obtidos e possíveis melhorias

16.1 Resultados obtidos

O projeto produziu uma biblioteca em C para grafos não direcionados e ponderados, organizada em módulos, com interface pública própria, duas formas de ligação, testes automatizados e histórico em Git. Os resultados medidos são:

Item	**Resultado**
Funções públicas	19, em 4 módulos (`core`, `edges`, `traversal`, `properties`)
Compilação	Sem avisos com `-Wall -Wextra` em todos os módulos
Biblioteca estática	`lib/libgraph.a`, 20 030 bytes
Biblioteca dinâmica	`lib/libgraph.so`, 20 976 bytes, com as 18 funções exportadas
Compilação incremental	A alteração de um `.c` recompila só o seu módulo; a de `core.h` recompila os quatro
Testes	59 asserções, todas aprovadas; 15 das 19 funções testadas com valores esperados
Git	76 commits (52 de desenvolvimento e 24 de merge), 10 branches de trabalho e integração por Pull Request

Em termos de arquitetura, a separação por tipo de pergunta (ciclo de vida, arestas, percursos e propriedades) manteve cada módulo coeso. As funções auxiliares ficaram restritas aos arquivos `.c`, e o cliente depende apenas de `graph.h`. Do ponto de vista do processo, a divisão em branches permitiu que os integrantes trabalhassem em módulos diferentes ao mesmo tempo, e a integração frequente com `-Wall` ativado expôs cedo os erros de tipos e de memória.

16.2 Possíveis melhorias

Interface e projeto.

- **Distinguir erro de ausência de aresta.** `graph_get_weight` devolve 0 tanto para aresta inexistente quanto para erro. Um código de retorno separado (por exemplo, devolver o peso por ponteiro e um booleano) eliminaria a ambiguidade.

Desempenho.

- **Lista de adjacência.** Para grafos grandes e esparsos, trocar a matriz por listas de adjacência reduziria a memória de $O(n^2)$ para $O(n + m)$ e aceleraria os percursos.
- **Heap no Dijkstra.** Com lista de adjacência e uma fila de prioridade, o custo cairia de $O(n^2)$ para $O((n + m) \log n)$.
- **Testes de desempenho.** Medir os tempos das funções em grafos de tamanhos crescentes para comparar as escolhas atuais com as alternativas.

Qualidade e automação.

- **Integração contínua** com GitHub Actions executando `make` e `make test` a cada Pull Request.
- **Documentação gerada** com Doxygen, aproveitando os comentários já escritos nos cabeçalhos.
- **Depuração com GDB**, aproveitando a opção `-g` em um alvo `debug` do `Makefile`.
- **Novas funcionalidades**, como árvore geradora mínima, grafos direcionados e leitura do grafo a partir de arquivo.